A WORKING METHOD

How I code with Claude Code.

Foundation first. Code second. Why two hours of writing markdown saves you weeks of drift.

Aurono Labs · auronolabs.com · 2026

THE IDEA

One sentence to remember.

> I don't tell Claude Code what to build.
> I tell it what we stand for, what we're shipping,
> how we test, and how we ship — then building becomes
> the easy part.

— the working method

PROBLEM

Long projects drift.

A weekend hack survives without structure. A real product doesn't. Across hundreds of sessions you lose coherence: vocabulary, scope, design choices, what "done" means.

VOCABULARY

"Trade" ≠ "Order" ≠ "Fill"

Two engineers using the same word for different things is where rot begins.

SCOPE

Phase 2 leaks into Phase 1

Without a roadmap, every shiny idea looks like the next sprint.

QUALITY

"Done" without tests

Without a workflow, commits land at 11pm with no functional verification.

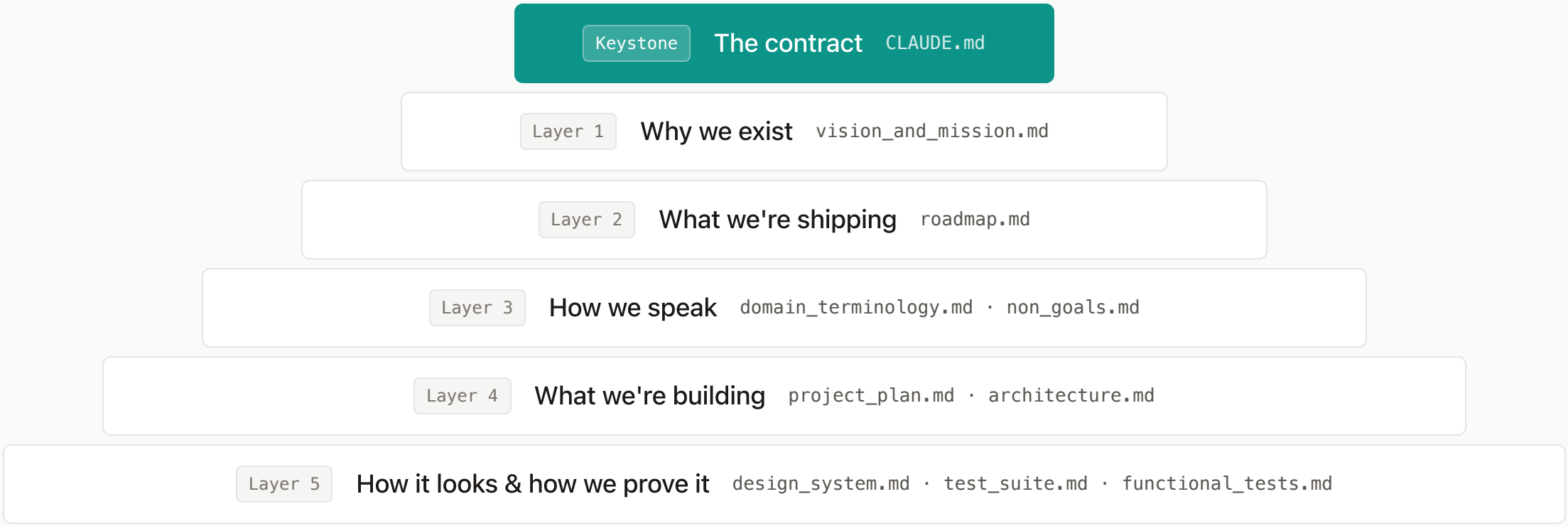
The fix isn't a smarter model. It's **a written foundation the model re-anchors on every session.**

THE FOUNDATION

Five layers and a keystone.

Aurono Labs

Each layer constrains the one below it. The keystone — `CLAUDE.md` — points to every layer and defines how the work flows through them.



Why we exist.

→ docs/vision_and_mission.md

What it decides

Every feature can be tested against this. If it can't be reconciled with the mission, it doesn't get built.

The 130 lines that decide everything else.

- "Local-first, execution-only. No custody, no predictions, no centralized control."
- The open-source / proprietary boundary is drawn here, once.
- The business model is here too — so feature ideas that break it are obvious early.

FROM AURONO START → docs/vision_and_mission.md

"If software can move money, it must be locally controlled, fully inspectable, incentive-aligned. Anything else eventually breaks trust."

Not advice. Not custody. Not prediction.

What we're shipping.

→ docs/roadmap.md

What it decides

The difference between `now`, `later`, and `never`. Phase 2 features stop sneaking into Phase 1 work.

Phases with explicit scope.

- Each phase has a focus, a date window, and a clear boundary.
- For every phase: what's open source, what's proprietary, and why.
- The roadmap and the vision agree — or one of them is wrong.

FROM AURONO START → docs/Aurono_Roadmap.md

Phase 1	Aurono Start	2026 Q1	Local execution-only crypto trading
Phase 2	Aurono Portfolio	2026 Q2	Portfolio-aware execution
Phase 3	Strategy Store	2026 Q4	Disclosed strategies, user keeps control
Phase 4	Multi-Asset	2027	Beyond crypto
Phase 5	Infrastructure	parallel	Embeddable execution layer

The locked language.

→ docs/domain_terminology.md

→ docs/non_goals.md

The rule

Terms are **write-once**. Never renamed. Never overloaded. If a new concept appears, you introduce a new term.

Words are infrastructure.

- `Decision` ≠ `Intent` ≠ `Trade` ≠ `Order` ≠ `Fill`
- `Reservation` ≠ `Allocation`. The non-goals file lists what we explicitly will *not* do.
- The cost: 20 minutes once. The payoff: weeks of avoided ambiguity in code, events, UI, docs.

> **FROM AURONO START** → docs/domain_terminology.md — § Forbidden Substitutions
>
> ```
> allocation ≠ reservation
> execution  ≠ decision
> order      ≠ trade
> intent     ≠ decision
> trade      ≠ round trip
> ```
>
> "If you feel the urge to reuse a term for a new meaning, you need a new term instead." — v1.0, locked.

The plan, and the structure.

→ docs/project_plan.md

→ docs/architecture.md

What it decides

Single source of truth for what's done (✅), what's pending (⬜), what's in the backlog, what's parked.

Read every session start.

- The plan: phases, items, completion markers, backlog, parked. Append-only history.
- The architecture: technical decisions with their reasons. New options can't break old ones silently.
- New session? Claude Code already knows where we are. No re-briefing.

```
FROM AURONO START → docs/project_plan.md

PHASE 0 — Freeze Reality (Before Writing Any Code) ✅
  ✅ Document actual PoC behavior (not intended behavior)
  ✅ Declare compatibility boundaries — docs/v1_non_goals.md

PHASE 1 — Canonical Contracts (No Runtime Code Yet) ✅
  ✅ Finalize Aurono Event Taxonomy v1.0 (frozen)
  ✅ Define event payload schemas
```

LAYER 5

How it looks, how we prove it.

→ docs/design/design_system.md

→ docs/design/user_journeys.md

→ docs/testing/test_suite.md

→ docs/functional_tests.md

The senses, and the proof.

- Design system: colors, typography, accessibility floor (WCAG 2.1 AA). One register per surface.
- User journeys: who does what, when. Features map to journeys, not the other way around.
- Test suite: every test file documented — what it covers, why it exists.
- Functional tests: `open` → `passed`/`failed`. A Phase 4 deliverable, not an afterthought.

FROM AURONO START → docs/design/DESIGN-SYSTEM.md — § Color-Blind Safety

- Red `#DC2626` means broken — system errors only, never market fluctuations.
- Losses use warm sand / terracotta `#9B6549` — calm, not alarming.
- CVD-safe chart pair: Teal `#0D9488` + Slate `#94A3B8`.
- Multi-channel encoding: color + shape (▲ ▼) + explicit sign (`+€234 / −€12`) + label.
- Monospace for every number, price, quantity, ID. WCAG 2.1 AA is the floor.

KEYSTONE

CLAUDE.md — the contract.

The file Claude Code reads **first, every session**. It points to every foundation layer and defines how the work flows through them.

project conventions

Running, installing, testing

One table each. The commands every contributor needs in the first five minutes.

the 6-phase workflow

Five trigger phrases

Idea → Backlog → Plan → Execute → Test → Close. Each phase has exact phrasing the operator uses.

discipline

"Discuss before building"

Questions are not requests to code. Plans are written before code is touched. Master is sacred.

domain alignment

Pointer to every doc

Vision, terminology, design — each constraint travels into every session via this one file.

git rules

Feature branches, not master

Pre-commit hook enforces functional-test updates. The workflow has teeth, not just text.

review protocol

Foundation stays alive

Manual + automatic checks for drift between files. Monthly reconvergence is part of the contract.

THE METHOD

For each layer: have a conversation.

Don't *drop* these files in the repo and hope. For each one, work it through with Claude Code until the doc is strong enough to push back on bad ideas.

1

Read it together

"Read `vision_and_mission.md`. Tell me what you think it commits us to."

2

Stress-test it

Propose a bad idea. Watch it push back. If it doesn't, the doc isn't strong enough — fix the doc.

3

Lock it

Add a sentence to `CLAUDE.md`: "all work must align with X, defined in `<file>`."

4

Move to the next layer

Vision → Roadmap → Terminology → Plan → Design → Tests → Workflow.

Total cost: roughly **two hours of focused conversation per project**. Payback: within the first week.

THE WORKFLOW

Six phases, five triggers.

Once the foundation is in place, daily work becomes almost mechanical. Each trigger is a short phrase the operator types; the model knows what to do next.

PHASE 1

Idea Intake

`"New idea: …"`

PHASE 2

Backlog or Park

`"Add to backlog"`

PHASE 3

Planning

`"Let's build X"`

PHASE 4

Execution

`"Plan accepted"`

PHASE 5

Functional Testing

`(operator tests)`

PHASE 6

Closing

`"FTs closed"`

No drift. No *"wait, what were we building again?"* No commits to master at 11pm with no tests.

IN PRACTICE

What an idea looks like, end to end.

Aurono Labs

```
user: New idea: add a portfolio rebalancer.
claude: → structured assessment: vision alignment / value / effort / complexity / recommendation

—

user: Add to backlog.
claude: → appends to project_plan.md, shows the line

— weeks later —

user: Let's build the portfolio rebalancer.
claude: → full plan: modules, steps, pytests, functional tests, architectural decisions

—

user: Plan accepted.
claude: → writes code, writes tests, appends FTs (status: open), updates docs — does NOT commit

—

user: Functional tests closed.
claude: → feature branch, commit, push, updates USER-JOURNEYS if needed
```

Topics I can go deeper on.

The foundation is the starting point. Four follow-up topics that build on it — each worth its own session.

foundation · my favorite

Sharpening vision & mission

From vague intentions to a document that rejects bad ideas on its own. The stress-test method. What separates 130 lines that actually steer from five pages no one opens.

deployment

Website + Vercel

How the marketing site lives in the same repo, how Vercel auto-builds every push, and how per-branch preview URLs make the human check before merge nearly free.

discipline

The six phases — and what breaks without them

Per phase: why the trigger phrase exists. And what you get when you skip it: direct commits and pushes, no human-in-the-loop test, regressions that only surface in production.

maintenance

Periodic review of the foundation

Documents age silently. Monthly review baked into the workflow — not a loose intention. Without it, the contract loses its teeth within a quarter.

Interested in any of these? **Say the word and we'll book a follow-up.**

Foundation first.
Code second.

The model writes the code.

The documents write the model.

Aurono Labs · auronolabs.com · 2026

eppo@auronolabs.com

WHAT ELSE

Skills — instructions on demand.

A skill is a markdown package with YAML frontmatter that Claude Code auto-loads when its trigger fires. Conditional — unlike `CLAUDE.md`, which is always in context.

What they add

- **Conditional loading** — active only when the trigger fits. Less noise, sharper attention.
- **Enforcement** over statement — scan diffs, refuse violations, not just "remember to."
- **Slash commands** instead of fragile trigger phrases — unambiguous, with autocomplete and validation.

Example: six trigger phrases, six commands

"New idea: …"	**/idea**
"Add to backlog"	**/backlog**
"Let's build X"	**/plan**
"Plan accepted"	**/execute**
(operator test)	**/ft-cycle**
"FTs closed"	**/close**

> **Not a big win for me yet.** Discipline and `CLAUDE.md` already do a lot. I review this monthly — once a skill genuinely moves the needle, that's when I'll implement it.

WHAT ELSE

Plugins — beyond Claude's own world.

A plugin gives Claude Code access to tools it doesn't have on its own — a browser, a database, an external API. Skills are *instructions*; plugins are *capabilities*.

concept

What is a plugin?

An external program that exposes its tools to Claude Code via the MCP protocol. Configured once, then callable like any other tool. Configuration lives in `.claude/settings.json`, so the whole environment travels with the repo.

in use

Playwright — design in the browser

The plugin I'm using now. Claude opens the site in a real browser, clicks around, takes screenshots, and compares the result against `design_system.md`. The first design check without me looking at it myself.

WHAT THIS CAN BECOME FOR AURONO

- User journeys (login, order placement, fee preview) running headless in Phase 4 of the workflow.
- Exchange API plugin (Binance, Coinbase) for live contract checks on order flow.
- Postgres plugin to verify event taxonomy and payloads against real data.
- Linear or GitHub plugin to wire Phase 1 (idea intake) directly into ticketing.